

100 Days of Azure DevOps

Day 28 Handout — Stages remember order. Matrix clones the job. A deploy stub that ignores Test is theater

3 - Continuous Integration · Personal lab only · Educational content · Not a sales pitch

Architecture A — YAML objects and what they actually mean

Object	Owns	Typical keys
stage	A gate in the workflow	dependsOn, condition, jobs, variables
job	One agent lease	pool, strategy.matrix, steps, dependsOn (other jobs)
deployment	CD job + environment history	environment, strategy.runOnce.deploy (Day 31+)
matrix	N clones of the same job	strategy.matrix: py311 / py312 → \$(python.version)
condition	Whether the stage/job runs	succeeded() default after dependsOn; always() is explicit

Architecture B — dependsOn vs hope

Intent	YAML	If you omit it
Test after Build	stage Test dependsOn: Build	Stages can race; Test may start on a red Build
Stub only if green	condition: succeeded()	A failed test still 'deploys' an echo
Fan-out versions	strategy.matrix on the job (not the stage)	You tested one runtime and marketed three
Fan-in after matrix	dependsOn: Build condition: succeeded()	Publish runs while one matrix leg is still red

Architecture C — pitfalls that look parallel and lie

Pitfall	What you see	Fix
Matrix on a stage	Schema error or one job anyway	strategy lives on job
No parallelism SKU	Matrix jobs queue one-by-one	Expected on free Microsoft-hosted; still correct YAML
condition: succeededOrFailed()	Publish runs after Test exploded	Use that only when you mean cleanup
dependsOn: []	Stage starts with CI, skips the chain	Empty dependsOn is 'run now', not 'I forgot'

One-liner to remember

Matrix builds clone you across versions so 'works on my Node' becomes 'works on these Nodes' — and succeeded() keeps the stub honest.

Step-by-step lab (personal Azure / Azure DevOps only)

Personal Azure DevOps org only. Project azure-100-labs. Reuse Day 26 Python or Day 25 Node — do not rebuild a third app.

- 1 Open pipelines/python-ci.yml (or node). Split it into stages Build and Test. Test must set dependsOn: Build.
- 2 On the Build (or Test) job add strategy.matrix for two versions (3.11/3.12 or Node 18.x/20.x). NodeTool@0 / UsePythonVersion@0 reads \$(version).
- 3 Add stage Publish with dependsOn: Test and condition: succeeded() and a single script: echo stub — not a real deploy.
- 4 Run on main or a PR. Confirm the run graph shows two matrix legs, then Publish.

- 5 Break a test on purpose (assert 1 == 0), push, watch Publish stay skipped. If it still runs, the condition is theater — fix it.
- 6 Revert the broken test. Write one paragraph in docs/multistage-day28.md: dependsOn vs condition vs matrix.

Starter YAML — matrix + dependsOn + succeeded()

```
# pipelines/multistage.yml
trigger:
- main
stages:
- stage: Build
  jobs:
  - job: BuildJob
    pool: { vmImage: ubuntu-latest }
    strategy:
      matrix:
        py311: { version: '3.11' }
        py312: { version: '3.12' }
    steps:
    - task: UsePythonVersion@0
      inputs: { versionSpec: $(version) }
    - script: python --version
      displayName: Prove matrix version
- stage: Test
  dependsOn: Build
  condition: succeeded()
  jobs:
  - job: TestJob
    pool: { vmImage: ubuntu-latest }
    steps:
    - script: echo Tests would run here
- stage: Publish
  dependsOn: Test
  condition: succeeded()
  jobs:
  - job: Stub
    pool: { vmImage: ubuntu-latest }
    steps:
    - script: echo Stub only – no deploy bits
```

Done checklist

- Can point at dependsOn and condition: succeeded() in your YAML
- Matrix ran two versions (or you noted hosted parallelism queued them)
- A failed Test skipped Publish once
- docs/multistage-day28.md exists on the personal repo

Tomorrow — Day 29

Variables, groups, and secrets — runtime \$(), compile-time \${ }, and why secrets in YAML are postcards.